

Laboratory Work 4

Cross cutting concerns with Spring AOP

Course: RWPSF 3305 - Web Application Development with Spring Framework

Student: Turgynbek Amirbek

Group: IS2413

Class-list position: 20

Individual assignment: Variant 8 - call tracing

The variants repeat after 12, so position 20 corresponds to variant 8: $((20 - 1) \bmod 12) + 1 = 8$.

Aim

To move method logging, execution timing and user-action auditing into aspects, compare advice order, inspect the Spring proxy, and demonstrate why self-invocation bypasses interception. Variant 8 adds nested call tracing using Before and After advice.

Repository: <https://github.com/neuros1s/springlab2>

Target branch: <https://github.com/neuros1s/springlab2/tree/lab04>

Execution status: the code and checks are prepared; runtime results have not yet been captured. The branch link is the intended upload location.

Build and project

The Maven project includes the Lab 2 and Lab 3 classes and adds the Lab 4 service, controller, annotation and aspects. Spring Boot 3.5.16 manages the AOP starter dependency. The compiler targets Java 25, as required by the assignment.

```
mvn clean verify
java -jar target/spring-lab-01-0.0.1-SNAPSHOT.jar
```

Pointcuts and audit annotation

Pointcuts

```
package kz.iitu.springlab.aspect;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;
@Aspect
@Component
public class Pointcuts {
    @Pointcut("within(kz.iitu.springlab.service..*)")
    public void serviceLayer() {}
    @Pointcut("execution(public * *(..))")
    public void publicMethod() {}
    @Pointcut("serviceLayer() && publicMethod()")
    public void serviceOperation() {}
}
```

Audited

```
package kz.iitu.springlab.audit;
import java.lang.annotation.*;
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Audited {
    String action();
    boolean logArguments() default false;
}
```

serviceLayer selects methods declared in the service package and its subpackages. publicMethod limits interception to public methods. serviceOperation combines both conditions. The aspects reuse this named pointcut. The audit aspect has one annotation-binding expression so it can receive the Audited instance.

RUNTIME retention makes the annotation available to Spring while the application is running. action identifies the operation; logArguments controls whether the argument values are recorded.

Logging advice

LoggingAspect

```
package kz.iitu.springlab.aspect;
import java.util.Arrays;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(2)
public class LoggingAspect {
    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);
    @Before("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public void before(JoinPoint jp) {
        log.info("[LOG] -> {} args={}", jp.getSignature().toShortString(),
            Arrays.toString(jp.getArgs()));
    }
    @AfterReturning(pointcut="kz.iitu.springlab.aspect.Pointcuts.serviceOperation()",
        returning="result", argNames="jp,result")
    public void returned(JoinPoint jp, Object result) {
        log.info("[LOG] <- {} returned {}", jp.getSignature().toShortString(), result);
    }
    @AfterThrowing(pointcut="kz.iitu.springlab.aspect.Pointcuts.serviceOperation()",
        throwing="ex", argNames="jp,ex")
    public void failed(JoinPoint jp, Throwable ex) {
        log.info("[LOG] !! {} type={} message={}", jp.getSignature().toShortString(),
            ex.getClass().getSimpleName(), ex.getMessage());
    }
}
```

Before records the method and arguments. AfterReturning records a successful result. AfterThrowing records the exception type and message. Its throwing name matches the ex parameter, and returning matches result. A successful call uses Before and AfterReturning; a failing call uses Before and AfterThrowing.

Execution timing

TimingAspect

```
package kz.iitu.springlab.aspect;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
@Aspect
@Component
@Order(3)
public class TimingAspect {
    private static final Logger log = LoggerFactory.getLogger(TimingAspect.class);
    @Around("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public Object measure(ProceedingJoinPoint pjp) throws Throwable {
        long started = System.nanoTime();
        try {
            return pjp.proceed();
        } finally {
            long ms = (System.nanoTime() - started) / 1_000_000;
            String method = pjp.getSignature().toShortString();
            if (ms > 200) log.warn("[TIME] SLOW: {} - {} ms", method, ms);
            else log.info("[TIME] {} - {} ms", method, ms);
        }
    }
}
```

System.nanoTime measures elapsed time. proceed executes the next interceptor or target method and preserves the return value. The finally block records time on both success and failure. The deliberately slow findAll method sleeps for 300 ms, so it should exceed the 200 ms warning threshold.

Auditing and advice order

AuditAspect

```
package kz.iitu.springlab.aspect;
import java.time.Instant;
import java.util.Arrays;
import kz.iitu.springlab.audit.Audited;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(1)
public class AuditAspect {
    private static final Logger log = LoggerFactory.getLogger(AuditAspect.class);
    @Around(value="@annotation(audited)", argNames="pjp,audited")
    public Object audit(ProceedingJoinPoint pjp, Audited audited) throws Throwable {
        String args = audited.logArguments()
            ? " args=" + Arrays.toString(pjp.getArgs()) : "";
        log.info("[AUDIT] start {} at={}", audited.action(), Instant.now(), args);
        try {
            Object result = pjp.proceed();
            log.info("[AUDIT] {} success at={}", audited.action(), Instant.now());
            return result;
        } catch (Throwable ex) {
            log.info("[AUDIT] {} failure at={} error={}", audited.action(),
                Instant.now(), ex.getMessage());
            throw ex;
        }
    }
}
```

AuditAspect has order 1, LoggingAspect order 2, and TimingAspect order 3. Among these three, audit enters first and exits last. Exceptions are logged and rethrown. The controller maps invalid identifiers to HTTP 400 only after the exception has passed through the aspects.

Position	Core record
1	AUDIT start
2	LOG arguments
3	TIME elapsed
4	LOG result
5	AUDIT success

Individual assignment call tracing

TracingAspect

```
package kz.iitu.springlab.aspect;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
@Aspect
@Component
@Order(0)
public class TracingAspect {
    private static final Logger log = LoggerFactory.getLogger(TracingAspect.class);
    private final ThreadLocal<Integer> depth = ThreadLocal.withInitial(() -> 0);
    @Before("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public void enter(JoinPoint jp) {
        int current = depth.get();
        log.info("[TRACE] {}-> {} depth={}", " ".repeat(current),
            jp.getSignature().toShortString(), current);
        depth.set(current + 1);
    }
    @After("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public void leave(JoinPoint jp) {
        int current = Math.max(0, depth.get() - 1);
        try {
            log.info("[TRACE] {}<- {} depth={}", " ".repeat(current),
                jp.getSignature().toShortString(), current);
        } finally {
            if (current == 0) depth.remove();
            else depth.set(current);
        }
    }
}
```

TracingAspect has order 0, so its entry and exit surround the three core aspects. Each thread has its own depth value. Before prints the current indentation and increments the depth. After runs on success and failure, reduces the depth, and removes the ThreadLocal value when the outermost call finishes. This avoids stale indentation on reused request threads.

The two spaces per level make proxy-visible nesting readable. Self-invoked methods are intentionally absent from this trace because they never pass through a proxy.

Service and self invocation fix

CatalogService

```
package kz.iitu.springlab.service;
import java.util.List;
import java.util.stream.IntStream;
import kz.iitu.springlab.audit.Audited;
import org.springframework.stereotype.Service;
@Service
public class CatalogService {
    public String findById(long id) {
        sleep(50);
        return "Item no. " + id;
    }
    @Audited(action="CATALOG_LIST", logArguments=true)
    public List<String> findAll(int limit) {
        if (limit < 1 || limit > 100) throw new IllegalArgumentException("Limit must be 1..100");
        sleep(300);
        return IntStream.rangeClosed(1, limit).mapToObj(i -> "Item no. " + i).toList();
    }
    @Audited(action="CATALOG_REMOVE")
    public String remove(long id) {
        if (id <= 0) throw new IllegalArgumentException("Invalid identifier: " + id);
        return "Removed item no. " + id;
    }
    public String removeTwice(long id) {
        String first = this.remove(id);
        String second = this.remove(id + 1);
        return first + "; " + second;
    }
    private void sleep(long ms) {
        try { Thread.sleep(ms); }
        catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("Operation interrupted", ex);
        }
    }
}
```

FixedRemovalService

```
package kz.iitu.springlab.service;
import org.springframework.stereotype.Service;
@Service
public class FixedRemovalService {
    private final CatalogService catalog;
    public FixedRemovalService(CatalogService catalog) { this.catalog = catalog; }
    public String removeTwice(long id) {
        String first = catalog.remove(id);
        String second = catalog.remove(id + 1);
        return first + "; " + second;
    }
}
```

CatalogService.removeTwice uses this.remove and retains the original problem. FixedRemovalService is a separate coordinating bean; its two calls reach the injected CatalogService proxy. This separates orchestration from removal and avoids a circular self reference.

Successful and failing requests

GET /api/lab4/items?limit=5 returns five items. The numbered core records show audit, logging and timing in the required relative order; the tracing records wrap them.

Expected observation: Five core records in order, plus tracing entry and exit.

Runtime evidence is pending. Run mvn clean verify and the report script to insert the actual output here.

DELETE /api/lab4/item/0 raises IllegalArgumentException. The aspects observe the failure and preserve it; the controller returns HTTP 400 with the type and message.

Expected observation: LOG exception, AUDIT failure, timing and tracing cleanup; no successful-return advice.

Runtime evidence is pending. Run mvn clean verify and the report script to insert the actual output here.

Proxy inspection and observations

Expected observation: CGLIB subclass of CatalogService; both AOP and CGLIB flags are true.

Runtime evidence is pending. Run mvn clean verify and the report script to insert the actual output here.

The runtime bean is a generated subclass, typically containing `$$$SpringCGLIB$$` in its name. Its superclass is CatalogService. Spring Boot uses class-based proxies by default; the suffix is generated infrastructure, not a class written by the student.

Call or measurement	Expected	Observed
External remove audit records	2	Pending
Self-invocation audit records	0	Pending
Self-invocation logging records	2	Pending
Fixed nested-call audit records	4	Pending
Fixed logging records	6	Pending

A pair of audit records is start plus outcome for one intercepted operation. The two logging records before the fix belong to removeTwice itself, not to the two remove calls. After the fix the three intercepted operations create three logging pairs.

Self invocation before the fix

GET /api/lab4/remove-twice/5

Expected observation: Only `CatalogService.removeTwice` is logged, timed and traced. There are no audit records for its internal remove calls.

Runtime evidence is pending. Run `mvn clean verify` and the report script to insert the actual output here.

A Spring proxy intercepts calls arriving through the proxy. Once execution enters the target, this refers to the target object; a neighbouring method call is a normal Java call. An annotation alone cannot make that internal call cross the proxy boundary.

Self invocation after the fix

GET /api/lab4/remove-twice-fixed/5

Expected observation: The coordinator has depth 0 and the two CatalogService.remove calls have depth 1. Each removal is audited.

Runtime evidence is pending. Run mvn clean verify and the report script to insert the actual output here.

The integration check also calls the fixed endpoint with identifier 0, then makes a fresh request. The error must unwind both trace levels and the next request must start at depth 0. This checks cleanup rather than only the normal path.

Defence answers

1 What is a cross cutting concern

A responsibility needed across multiple operations. In this project it is logging, timing, auditing and tracing.

2 Pointcut and advice

The Pointcuts methods answer where interception applies. The advice methods answer what action is executed at those join points.

3 Meaning of the package expression

`within(kz.iitu.springlab.service..*)` matches types in service and nested packages. The double dot includes subpackages; a star is a wildcard within the relevant name pattern.

4 Why two annotations

Aspect defines the aspect metadata. Component lets component scanning register an instance as a Spring bean.

5 Advice order within one aspect

For these advice types the entry side is Around then Before. A normal or exceptional outcome runs AfterReturning or AfterThrowing; After has finally semantics. Control then returns to the outer Around advice.

6 Removing proceed

The target is not called by that advice. The result is whatever Around returns; an exception occurs only if the advice itself throws.

7 Missing AfterReturning on failure

The method threw instead of returning normally, so AfterThrowing applies.

8 The throwing attribute

It binds the thrown exception to the advice parameter. A wrong parameter name prevents correct binding and normally causes aspect creation to fail.

Defence answers continued

9 Runtime annotation retention

Spring must inspect the annotation while the application runs.

10 Annotation binding

The annotation pointcut binds the matched Audited instance to the advice argument named audited.

11 Which aspect runs first

Tracing has order 0 and wraps everything. Among the three required aspects, audit has order 1 and enters first and closes last.

12 Generated class name

It identifies the CGLIB subclass used to intercept calls while delegating to the target implementation.

13 Why CGLIB

Spring Boot defaults to class-based proxies. Merely adding an interface does not override that Boot setting.

14 Self invocation and the fix

Internal calls bypass the proxy. A separate coordinator bean now calls the injected CatalogService proxy.

15 Private and static methods

Private methods cannot be overridden by a subclass proxy. Static methods belong to the class rather than a proxied instance. Spring proxy-based AOP cannot advise either.

16 Other annotations using proxies

Transactional, Async and Cacheable commonly use proxy-based interception when their corresponding support is enabled.

Conclusions

Aspects keep repeated operational concerns out of catalog methods. Named pointcuts make the interception boundary explicit, while Order makes nesting predictable. Proxy-based interception explains why internal calls behave differently from calls between beans. Separating the coordinator from the removal bean restores auditing and exposes nested calls to tracing.

References: Laboratory work No. 4 assignment; Spring Framework Reference, Declaring Advice and Proxying Mechanisms.